

Программирование и алгоритмизация

(МИСиС, осень-зима 2025)

Полевой Дмитрий Валерьевич
к.т.н., доцент КиК

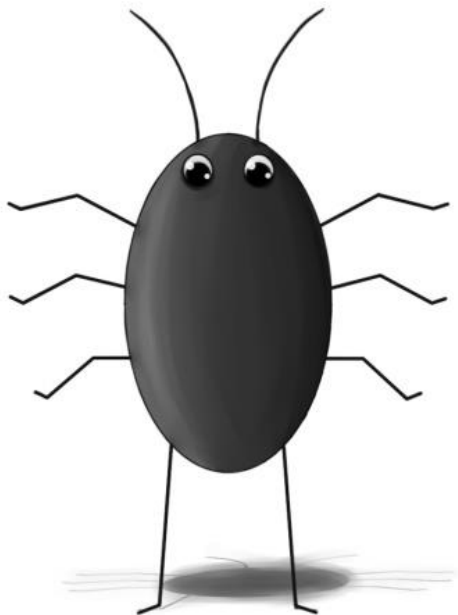
teams: 8url2s2

e-mail: dvpsun@gmail.com

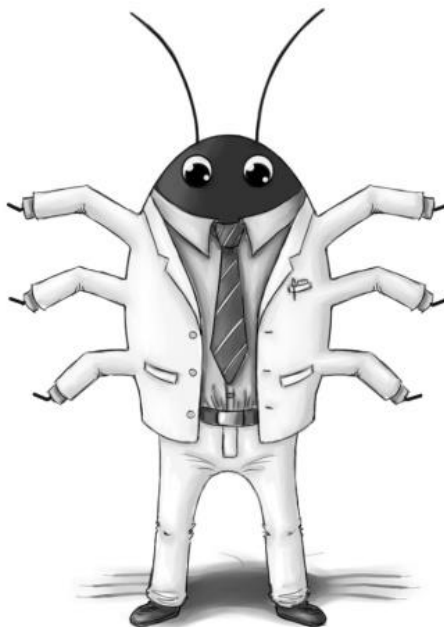
tg: @dvpsun

Это не баг, это фича...

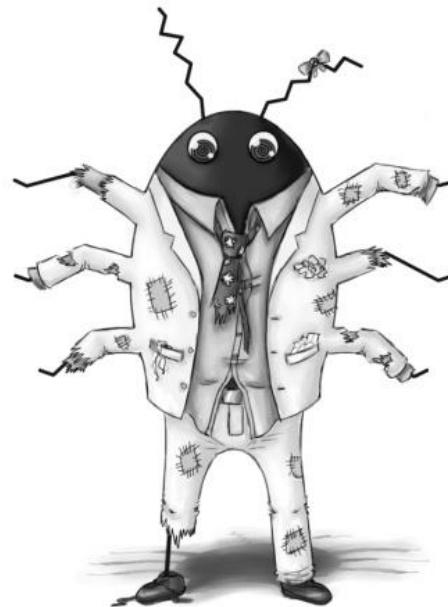
iris-hime.deviantart.com



BUG



FEATURE



BY DESIGN

Спецификация ПО

- точное описание (работы программы)
- корректность – согласованность с некоторой спецификацией
- ошибка – нарушение спецификации

Ошибки ПО и ошибки I/O

- неправильная реализация алгоритма
- некорректный вызов
(из другого фрагмента кода)
- некорректный обмен данными
 - ошибки пользователя
 - отказы оборудования (в т.ч. при выводе)
 - ошибки данных

Некорректный ввод и вывод

НЕ МОЖЕТ СЧИТАТЬСЯ ОШИБКОЙ

- реальный мир несовершенен 😊
- требуется
 - диагностика и подсказки пользователю
 - альтернативные варианты действий
 - предотвращение потери данных
 - ...

Как (вы)жить в неидеальном мире

- быть «немного параноиком»
- тестировать (больше и дешевле)
- изолировать подсистемы ввода/вывода
- обеспечивать корректные данные для алгоритмической части
- предусматривать схемы дублирования и резервирования критической информации

Действия в случае ошибок

- игнорировать
- сообщить
 - вывести в консоль
 - сохранить в протокол
 - послать сообщение разработчикам
 - и т.д.

Действия в случае ошибок

- прекратить исполнение программы
- вернуть статус ошибки
- вернуть допустимое значение и оставить программу в некорректном состоянии
- вызвать функцию для обработки ошибочных ситуаций

Глобальный статус (пример)

```
int getLastErr();  
void setLastErr(const int );  
  
if (ERR_NO == getLastErr()) {  
    // Основной поток исполнения программы.  
} else {  
    // Обработка ошибочной ситуации.  
}
```

Возвращаемое значение (пример)

```
int doJob();  
  
...  
if (ERR_NO == doJob()) {  
    // Основной поток исполнения программы.  
} else {  
    // Обработка ошибочной ситуации.  
}
```

Обработка ошибочных ситуаций

- глобальный статус
- возвращаемое значение
- на ком лежит обработка?
 - заказчик (клиент)
 - исполнитель (сервер)

возникновение ошибочной ситуации можно
просто игнорировать!

Защитное программирование

- программа всегда должна работать
- каждая функция защищается как может
- кто сломался, тот и виноват
- дублирование кода проверок
- сложность кода
- безответственное программирование

Программирование по контракту

- обязанности сторон
 - клиент: корректные входные данные
 - поставщик: корректная работа и результат
- соблюдение контракта
 - кто нарушил контракт, тот и виноват

Контрактные отношения

- могут быть между частями программы
программа-программа
- не определены с внешней средой
программа-человек
программа-внешний мир

Обработка входных данных

- архитектура
 - модули ввода (фильтры)
 - модули обработки
- гарантия корректности входных данных для обрабатывающих модулей

вопросы

9/9

Relay 2145
Relay 3370

1545

Relay #70 Panel F
(moth) in relay.

First actual case of bug being found.
antennae started.
closed down.

Контракты в исходном коде

- д.б. описаны в комментариях
- для проверки `assert (условие)`

пример:

```
#include <cassert>
```

```
int findBest(const Data* pData) {  
    /// если false, завершить работу программы  
    assert(0 != pData);  
    /// основной код  
}
```

Обработка исключений в C++

- модель с завершением
 - автоматическое продолжение программы невозможно
 - обработчик освобождает ресурсы и отдает управление на верхний уровень
- механизм обработки опциональный (опции компилятора)

Идеи обработки исключений

- разделить основной код и код обработки сбоев
- обрабатывать сбои на том уровне, на котором это может быть сделано
- усложнить игнорирование появления сбоев
- унифицировать способ передачи информации о сбое

Запуск исключений

- в системе создается объект с информацией об исключении, который «запускается» из текущего контекста
- «нормальное» исполнение программы прерывается и начинается «обработка исключения»

Источники исключений

- обработка аппаратных прерываний
- функции стандартной библиотеки
- функции «системных» библиотек
- пользовательский код

Исключение – что это такое

- может быть любого типа
(как пользовательского, так и встроенного)
- несет информацию (о проблеме)
 - имя класса (типа) исключения
 - данные экземпляра исключения
- создается и копируется в точке запуска

Запуск исключений (пример)

```
#include <stdexcept>

...
if (0 == denum) {
    throw std::invalid_argument(
        "Zero denominator in Rational "
        "ctor");
}

...
```

Локализация блоков с исключениями

- `try`-блок – составная инструкция, при выполнении которой могут генерироваться исключения
- `try`-блок сопровождается одним или несколькими блоками обработки исключений (`catch`-блоки)

Обработка исключений

- обработчик исключений
 - осуществляет перехват и обработку исключений
 - располагается сразу после блока `try`
 - начинается с ключевого слова `catch`
- может использовать «пойманное» исключение и извлекать из него информацию

Обработка исключений (схема)

```
try {  
    // код, который может  
    // генерировать исключения  
} catch (const Type1& ex) {  
    // обработка исключений типа Type1  
} catch (const Type2& ex) {  
    // обработка исключений типа Type2  
}  
// нормальное выполнение программы...
```

Раскрутка стека

- просмотр стека и поиск подходящего обработчика для исключения
- по мере раскрутки стека автоматически уничтожаются локальные объекты
- порядок разрушения обратный порядку создания (стек программы)

Когда объект создан?

- объект считается созданным после завершения работы конструктора
- составной объект считается созданным после создания всех входящих в него объектов
- массив считается созданным после создания всех его элементов

Автоматическое разрушение

- в процессе раскрутки стека производится только для созданных
- т.е. в массиве только для тех элементов массива, которые были созданы

Поиск подходящего обработчика

- просматриваются «ближайшие» обработчики в порядке их следования в исходном коде
- объект (ссылка на объект) исключения производного класса перехватывается обработчиком базового класса
- при запуске указателя поиск идет по стандартным правилам преобразований указателей

Перехват исключения (пример)

```
class Ex1 {};  
class Ex2  
    : public Ex1 {  
public:  
    Ex2(const Ex1&);  
};  
  
void f() {  
    throw Ex1();  
}
```

```
try {  
    f();  
} catch (const Ex2& ex) {  
    // ...  
} catch (const Ex1& ex) {  
    // ...  
}
```

Преобразования исключений

- обработчик объекта базового типа «усекает» исключение производного типа до базового
- автоматические преобразования одного типа исключений к другому не производятся

Как перехватывать исключения

рекомендуются константные ссылки

- избегаем копирования объекта исключения
- избегаем усечения до родительского типа
- гарантируем постоянство объекта исключения

Перехват любых исключений

- список аргументов (...)
- следует размещать в конце списка перехватчиков
- невозможно получить информацию об исключении и его типе

Перехват любых исключений (пример)

```
try {  
    // код  
} catch (const std::bad_alloc& ex) {  
    // обработка bad_alloc  
} catch (...) {  
    // обработка всего остального  
}
```

Перезапуск исключения

- передает исключение далее
- необходим, если текущий обработчик не может выполнить всю работу
- вызывается `throw` без аргумента внутри обработчика исключения

Перезапуск исключения (пример)

```
try {  
    // код  
} catch (const Ex1& ex1) {  
    // обработка  
    throw;  
} catch (...) {  
    // обработка  
    throw;  
}
```

Вызов `terminate`

- неперехваченное исключение
- исключение в деструкторе локального объекта в процессе раскрутки стека
- исключение в конструкторе или деструкторе глобального или статического объекта
- нарушение `noexcept` спецификации

Вызов `terminate`

- приостанавливает раскрутку стека
- по умолчанию вызывает `abort()` из системной библиотеки

ваши вопросы

Спецификация исключений

использовать запрещено

- часть объявления функции
- список типов исключений, которые могут быть сгенерированы функцией

пример:

```
void func(int a) throw(Ex1, Ex2);
```

Спецификатор `noexcept`

- применим к функциям (в т.ч. лямбда) и производным типам
- `noexcept` не является частью типа
- переопределяемые виртуальные функции должны иметь не менее строгую спецификацию исключений

Формы спецификатора `noexcept`

- `noexcept ("expression")`
- `noexcept` – аналогично `noexcept (true)`

пример:

```
void f() noexcept;
```

```
void (*pf)() noexcept(false);
```

Использование `noexcept`

- запуск исключения из `noexcept` функции приводит к `std::terminate()`
- аккуратное использование
 - ускорение работы алгоритмов (стандартных)
 - уменьшение объема исполняемого кода

Стандартные исключения

- образуют иерархию классов
- объявлены в пространстве имен `std`
- объявлены в заголовочных файлах
`<stdexcept>`, `<new>`, `<exception>`,
`<ios>`, `<typeinfo>`

Стандартные исключения (фрагмент иерархии)

`std::exception`

- `logic_error`

- `length_error` – неверная длина
- `out_of_range` – выход за пределы диапазона
- `invalid_argument` – неверный аргумент

- `runtime_error`

- `range_error` – ошибка диапазона
- `overflow_error` – переполнение

- `bad_alloc`

Где перехватывать исключения?

- там, где есть достаточно информации для обработки сбоя
- с целью
 - обработать сбой
 - преобразовать исключение
 - соблюсти границы подсистем

Гарантии безопасности исключений

- базовая гарантия
 - сбой может изменить состояние, не вызывает утечек и оставляет все объекты пригодными для дальнейшего использования
- строгая гарантия
 - сбой не изменяет состояния (транзакция)
- гарантия бессбойности
 - сбой не может произойти

Сбои и повторное использование

- устойчивость и цена разработки должны соответствовать решаемой задаче
- требования к устойчивости неизвестны
- сообщение пользователю
 - неизвестны доступные устройства вывода
 - неизвестно наличие пользователя

ваши вопросы

У БАГОВ ТОЖЕ ЕСТЬ ЧУВСТВА

НАЙДЯ БАГ:
СООБЩИТЕ О НЕМ

БАГИ НЕ ЛЮБЯТ
БЫТЬ ЗАБЫТЫМИ



НАЙДЯ БАГ:
ИЗУЧИТЕ ЕГО

БАГИ ЛЮБЯТ
БЫТЬ ПОНЯТЫМИ



НАЙДЯ БАГ:
СДЕЛАЙТЕ СНИМОК

БАГИ ЛЮБЯТ ХРАНИТЬ
ПАМЯТЬ О ВСТРЕЧЕ



НАЙДЯ БАГ:
УЗНАЙТЕ ЕГО ДРУЗЕЙ

БАГИ ОБЩИТЕЛЬНЫ



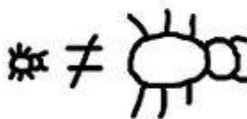
НАЙДЯ БАГ:
СООБЩИТЕ О НЕМ БЫСТРО

ИНАЧЕ БАГИ ОБЖИВУТСЯ И
ПОСТРОЯТ СЕБЕ ДОМА



НАЙДЯ БАГ:
БУДЬТЕ ЧЕСТНЫ

БАГИ НЕ ЛЮБЯТ
СПЛЕТЕН



НАЙДЯ БАГ:
ЗАПОМНИТЕ КАК ВЫ
ВСТРЕТИЛИ ЕГО

БАГИ РОМАНТИЧНЫ



НАЙДЯ БАГ:
НЕ ИГНОРИРУЙТЕ ЕГО

БАГИ МОГУТ УКУСИТЬ,
ЕСЛИ ИХ НЕ ЦЕНИТЬ



ваши вопросы

Модель памяти

- непрерывная память
 - адрес – сдвиг относительно «начала»
 - байт – единица адресации

0



Объект

- непрерывная область памяти
- трактуется как значение типа
- именованный или неименованный

пример:

```
int num(1);
```

```
p[15]
```

memsize типы

пригодны для хранения

- указателей
- индексов массивов (в т.ч. самых больших)

заголовок `<cstdint>`

`ptrdiff_t`

`size_t`

заголовок `<stdint.h>`

`intptr_t`

`uintptr_t`

Адрес

- целое неотрицательное число
- унарный оператор `&` – получение адреса

пример:

`&obj`

- `nullptr` – невалидный адрес
! не использовать макрос `NULL` и литерал `0`

Невалидный указатель

- гарантируется отсутствие объектов с адресом `nullptr`
- разыменование `nullptr` ошибка времени исполнения
- всегда проверять адрес перед использованием

Указатель (pointer)

- тип T^* – тип «указатель на T »
- переменная T^* содержит адрес объекта типа T
- унарный оператор $*$ – разыменование (косвенное обращение)

пример:

```
char c1 ( 'a' );
```

```
char* p(&c1); //< адрес переменной c1
```

```
char c2 (*p); //< c2 == 'a'
```

Массив (встроенный)

использовать по необходимости

- составной пользовательский тип `T [size]`
«массив из `size` элементов типа `T`»
- элементы индексируются от 0 до `size-1`

пример:

```
int divisors[12]{0};
```

Инициализация массива

- списком инициализаторов
- размер встроенного массива м.б. вычислен по списку инициализации
- не указанные в списке значения инициализируются по умолчанию
- используйте `T arr[SIZE]{0};`

пример:

```
int ar1[] {1, 2, 3, 4};  
int ar2[32] {1, 2};  
int ar3[MAX_NUMS] {0};
```

Указатели и массивы

- имя массива – указатель на первый элемент
- доступ через указатели или оператор []

пример:

```
p = &v[0]; //< или p = v;
```

```
v[i] == *(p + i)
```

Доступ к элементам массива

- указатель на массив и индекс

пример:

```
data[i]
```

- указатель на элемент массива

пример:

```
* (pData + i)
```

Сложение/вычитание указателя с целым числом

- «сдвиг» указателя (адреса) на заданное число элементов массива
- результат разыменования не определен, если адрес вне массива

пример:

```
pEnd = pBeg + n;
```

Вычитание указателей

- определено только для указателей на элементы одного массива (язык не позволяет быстро проверить)
- число (целое) элементов массива между указателями

пример:

```
ptrdiff_t n(pEnd - pBeg);
```

Указатель на void

- содержит адрес объекта
- допустимые операции
 - присвоение адреса
 - сравнение на равенство и неравенство
 - явное преобразование к указателю на другой тип
- запрещенные операции
 - разыменование
 - арифметические операции

Указатель на константу и константный указатель

- указатель на константную переменную
`const T* p(nullptr);`
- константный указатель (д.б. инициализирован)
`T* const pC(&obj);`
- константный указатель на константную переменную (д.б. инициализирован)
`const T* const pC(&obj);`

Объект, указатель или ссылка?

- если тип не фундаментальный, используйте ссылку или указатель (избегайте лишних объектов)
- предпочитайте ссылки
- если объекта может не существовать, используйте указатель
- помните о константности

ваши вопросы

Память (статическая, стек, куча)

- статическая память (при компиляции)
 - глобальные переменные
 - статические переменные
- стек
 - параметры функций
 - локальные переменные
- куча (heap)
 - динамическое распределение new/delete

Статические и глобальные объекты

- создаются «до начала работы `main`»
- уничтожаются «после завершения `main`»
- порядок для разных единиц трансляции не определен (static initialization order fiasco)
- используйте Construct On First Use Idiom (если очень сильно надо)

Стековые объекты

- область видимости → время жизни
- создаются в точке определения
 - локальные переменные
 - переменные циклов
 - передаваемые параметры функций
- уничтожаются при выходе из блока

Динамические объекты

- время жизни полностью определяется пользователем, а не ограничено областью видимости переменной
- «живут» в динамической памяти (куча)

Создание динамического объекта

`new T`

- оператор выделения памяти
- возвращает адрес объекта

пример:

```
int* pCounter(new int(0));
```

```
Complex* pZ(new Complex)
```

Примеры использования new

```
T* p0 (new T) ;
```

```
T* p1 (new T (x) ) ;
```

```
T* p2 (new (std::nothrow) T) ;
```

```
new (p2) T;
```

Ошибки выделения памяти

- запуск исключения `std::bad_alloc`
- `noexcept` версия `new` вернет `nullptr`

Уничтожение динамического объекта

`delete`

- оператор освобождения памяти
- указатель продолжает хранить адрес

пример:

```
delete pCounter;  
pCounter = nullptr;
```

Динамические массивы

использовать по необходимости

`new T[SIZE]` – оператор выделения памяти

`delete[]` – оператор освобождение памяти

пример:

```
int* pCounters(new int[NCOUNTERS]);
```

```
*(pCounters + 5) += 1;
```

```
delete[] pCounters;
```

```
pCounters = nullptr; //< обнулим для верности
```

Типовые проблемы (выход за пределы массива)

- «чтение мусора»
 - чтение из чужой памяти
(возможно, неинициализированной)
- «порча памяти»
 - запись в «чужую» память

проблемы могут проявиться
не сразу и не всегда

Типовые проблемы

(некорректные `new` и `delete`)

- утечка памяти
 - использование `new` без последующего `delete`
- «порча памяти»
 - вызов `delete` для уже освобожденной памяти или произвольного адреса

Практика использования `new` и `delete`

использовать только по необходимости

- всегда освобождать выделенную память
- инициализировать и зачищать указатели
- инкапсулировать в типы с `ctor/dtor`
- использовать умные указатели и контейнеры

ваши вопросы

Пространство имен (namespace)

- логически группирует объявления
- определяет область видимости
- может быть вложенным
- может быть анонимным
- открыто
 - допускается добавление имен в пространство в нескольких определениях

Создание пространства имен

```
namespace «имя_пространства_имен»  
{  
    // объявления и определения  
}
```

Создание пространства имен

```
namespace misis
{
    // Узнать число групп.
    int getGroupsCount();
}
```

Создание пространства имен (пример вложенности)

```
namespace misis
{
    namespace job
    {
        int getGroupsCount();
    }
}
```

Добавление имен

(пример открытости)

```
// in parser.h  
namespace parser {  
    bool parse();  
}
```

```
// in lexparser.h  
namespace parser {  
    int getLexCnt();  
}
```

Доступ к именам в пространстве

доступ через оператор разрешения области
видимости ::

имя_пространства::имя

пример:

std::cout

std::filesystem::path

Глобальное пространство имен

- все, что не лежит в собственном пространстве имен
- доступ через :: (можно и нужно опускать)

using-директива (избегать)

вносит все имена из заданного пространства имен
в текущую область видимости

```
using namespace имя_пространства_имен;
```

пример:

```
using namespace std;  
cout << "Hello!" << endl;
```

using-объявление (избегать)

ВВОДИТ имя в текущую область видимости

```
using  
    имя_пространства_имен::имя_члена;
```

пример:

```
using std::cout;  
  
cout << "Hello!" << std::endl;
```

using-псевдоним

объявляет псевдоним типа

- обычно, шаблонного
- м.б. шаблоном

```
using псевдоним_типа = тип;
```

```
template <class T>
```

```
using псевдоним_типа = тип;
```

using-псевдоним

(примеры использования)

```
using Vec =  
    std::vector<int>;
```

```
Vec v;
```

```
template<class T>  
using Ptr = T*;
```

```
Ptr<int> x;
```

Соккрытие имен

```
int x(10); // глобальная переменная
```

```
void f() {
```

```
    int x(0); // скрывает глобальную
```

```
    x = 1; // присваивание локальной
```

```
    ::x = 5; // присваивание глобальной
```

```
}
```

Анонимное пространство имен

ограничение видимости для компоновщика

```
namespace  
{  
    // локальный для единицы трансляции код  
}
```

Рекомендации по использованию

- в рамках курса – избегать `using`
- избегать `using` в заголовочных файлах
- использовать `using`-псевдонимы
 - в файлах реализации (ограниченно и разумно)
 - по необходимости внутри функций/типов

ваши вопросы

Абстрактный тип данных (АТД)

- описывает набор операций
(логика использования)
 - создание/уничтожение
 - изменение
 - получение описания

примеры АТД:

число, строка, массив, стек, очередь

Структура данных

- способ организации и манипуляции данными (в памяти)
- реализация АД (в исходном коде)

примеры структур данных:

кортеж, массив, список, дерево, граф

Интерфейс

- спецификация услуг
 - семантический уровень (проектирование)
 - синтаксический уровень (исходный код)
- в узком смысле — набор всех функций
 - для типа
 - для модуля
 - для подсистемы

Инвариант типа

- **состояние экземпляра**
 - значения всех переменных
 - может изменяться методами
 - может изменяться прямым изменением полей
- определяется типом
(логика использования)

Инвариант типа

- **должен выполняться**
в начале и конце каждого метода
- может контролироваться
assert, специальные методы

АТД «число»

- создание, уничтожение, копирование
- сравнение ($=$, $\neq$, $<$, $\leq$, $>$, $\geq$)
- арифметические операции
- $-$, $+=$, $+$, $-$, $*=$, $*$, $/=$, $/$
- ВВОД и ВЫВОД ($<<$, $>>$)

АТД «комплексное число»

$$z = \{re, im\} \quad \text{или} \quad z = re + i * im$$

$re \in \mathbb{R}$ - вещественная часть

$im \in \mathbb{R}$ - мнимая часть

$i^2 = -1$ - мнимая единица

инварианта нет

читать: Босс В. Лекции по математике: ТФКП (Т.09)

АТД «рациональное число»

$$r = \frac{num}{denum}, \text{ где}$$

$num \in \mathbb{Z}$ -целое и $denum \in \mathbb{N}$ - натуральное

$$\text{НОД}(num, denum) = 1$$

числитель и знаменатель не могут принимать
любые (целые) значения, т.е. существует инвариант

ваши вопросы

Матрица (в математике)

математический объект, записываемый в виде прямоугольной таблицы чисел и допускающий алгебраические операции (сложение, вычитание, умножение) между ним и другими подобными объектами. Обычно матрицы представляются двумерными (прямоугольными) таблицами.

а что такое таблица?

$f(0,0)$	$f(0,1)$	$f(0,2)$
$f(1,0)$	$f(1,1)$	$f(1,2)$
$f(2,0)$	$f(2,1)$	$f(2,2)$

Матрица (математически)

$$f: X \times Y \rightarrow C$$

где

$x \in X = [0, M - 1]$ - индекс строки

$y \in Y = [0, N - 1]$ - индекс столбца

отображение из «пространства индексов»
(полное декартово произведение)
в пространство «значений элементов»

АТД «матрица»

- создание, уничтожение, копирование
- получение доступа к элементу по индексу (iRow, iCol)
- получение размера по измерению
- изменение размера по измерению
- печать

Операции

- алгебраические (соответствующие размеры)
 - умножение матрицы на число
 - сложение двух матриц
 - умножение двух матриц
- поэлементные (одинаковые размеры)
 - сложение
 - умножение и деление

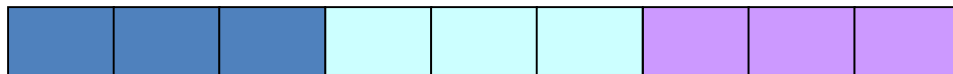
Дополнительные операции

- транспонирование
- срезы
- вставка/удаление столбца/строки
- перестановка двух строк/столбцов
- сложение двух строк/столбцов
- умножение на число строки/столбца

Приведенный индекс

- размеры матрицы
- массив элементов матрицы

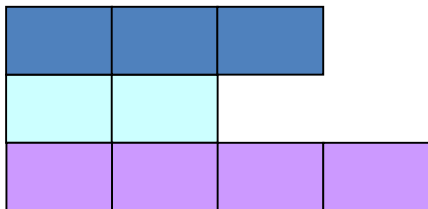
$$idx = i * N + j$$



почему
«массив массивов»
это не
«матрица»?

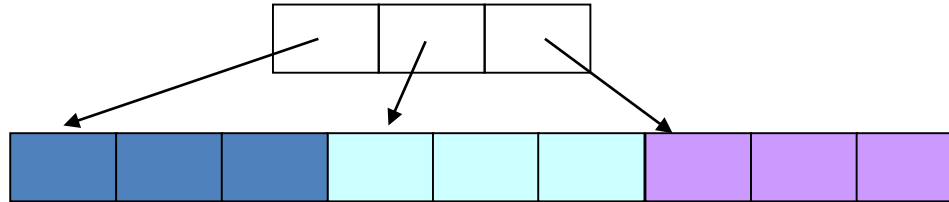
Массив массивов

- «ожерелье» (Jugged Array)
- элементы могут иметь разную длину



Массив строк/столбцов

- массив элементов матрицы
- массив индексов начала строк/столбцов



какое представление лучше?

Оценка реализации

- зависит от типового набора операций и распределения частот операций
- в массиве строк/столбцов «быстрая» перестановка строк/столбцов

ваши вопросы